অধ্যায় ২৫

২৫

JavaScript Backend Development with Node.js: REST APIs, Express, Databases, Authentication এবং Production Architecture

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Browser-এর JavaScript ও Server-এর Node.js-এর দায়িত্ব ও ঝুঁকির পার্থক্য বুঝতে।
- REST নীতিতে Resource, HTTP Method, Status Code ও URL নকশা করতে।
- Express-এ Route, Router, Middleware Chain এবং Error-handling Middleware লিখতে, এবং Express 4-এর Async Error-এর ফাঁদ এড়াতে।
- Route, Service ও Repository-তে ভাগ করা Layered Backend তৈরি করতে।
- Bearer Token দিয়ে Authentication এবং Role ও মালিকানাভিত্তিক Authorization প্রয়োগ করতে।
- Cursor Pagination, Filtering, `Idempotency-Key` ও `If-Match`-ভিত্তিক Optimistic Concurrency প্রয়োগ করতে।
- Request ID, Structured Log, Rate Limit, Health ও Readiness Check এবং Graceful Shutdown যোগ করতে।
- Repository-র চুক্তির পেছনে কীভাবে একটি আসল Database বসে, তা বুঝতে।
- একটি সম্পূর্ণ পরীক্ষিত **LeadDesk API** তৈরি করতে।

পূর্বশর্ত

- অধ্যায় ১৫: `fetch`, HTTP-এর মৌলিক ধারণা।
- অধ্যায় ১৬ ও ২২: `node:test`, Integration Test।
- অধ্যায় ২০: `node:http`, Validation, Authentication বনাম Authorization, Log Redaction।
- অধ্যায় ২১: Layered Architecture, Dependency Injection।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি ভ্রমণ সংস্থার Website-এর "যোগাযোগ করুন" Form থেকে প্রতিদিন শত শত জিজ্ঞাসা (Lead) আসে: কে, কোথায় যেতে চান, কতজন। বিক্রয় দলের প্রতিজন Agent নিজের Lead-গুলো দেখেন ও এগিয়ে নেন, Manager সবকিছু দেখেন ও ভাগ করে দেন। এই তথ্যের কেন্দ্রে একটি **Backend API**: Website, Mobile App ও অফিসের Dashboard—সবাই এর সঙ্গে কথা বলে।

Backend-এর ভুল Frontend-এর ভুলের চেয়ে অনেক বেশি ক্ষতিকর। একটি Bug একজন ব্যবহারকারীর পর্দায় নয়, সবার তথ্যে প্রভাব ফেলে; একটি ধীর Endpoint পুরো Server আটকে দেয়; একটি Authorization-এর ফাঁক সব গ্রাহকের ফোন নম্বর ফাঁস করে। অধ্যায় ২০-এ আপনি `node:http` দিয়ে নিরাপত্তার মূল নিয়ন্ত্রণগুলো তৈরি করেছেন। এই অধ্যায় তার ওপর পেশাদার Backend-এর কাঠামো যোগ করে: Express, REST-এর নকশা, এবং এমন নির্ভরযোগ্যতার কৌশল, যা Network-এর অনিশ্চয়তায় ও একসঙ্গে অনেক ব্যবহারকারীর মধ্যে তথ্য সঠিক রাখে।

২৫.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
`node:http`-এ হাতে Routing (অধ্যায় ২০)	Express: Router, Middleware Chain, Error Middleware
Cookie Session ও CSRF (অধ্যায় ২০)	API-র জন্য Bearer Token, Role ও Permission
404 দিয়ে অন্যের তথ্য লুকানো (অধ্যায় ২০)	Agent বনাম Manager-এর পূর্ণ অনুমতির নিয়ম
Retry (অধ্যায় ১৬, ২২)	Idempotency-Key: Retry-কে নিরাপদ করা
Immutable হালনাগাদ (অধ্যায় ১৮)	Version ও `If-Match`: একসঙ্গে দুজনের হালনাগাদ
Redaction-সহ Log (অধ্যায় ২০)	Request ID, প্রতিটি অনুরোধে একটি JSON Log
—	REST-এর নকশা, Cursor Pagination, Health/Ready, Graceful Shutdown

২৫.২ Browser JavaScript বনাম Node.js Server

একই ভাষা, ভিন্ন পৃথিবী:

বিষয়	Browser	Node.js Server
কার যন্ত্রে	ব্যবহারকারীর	আপনার নিয়ন্ত্রিত Server-এ
বিশ্বাস	কোনো কিছুই নয় (ব্যবহারকারী Code বদলাতে পারেন)	নিজের Code; কিন্তু প্রতিটি অনুরোধ অবিশ্বস্ত
একসঙ্গে কতজন	একজন	শত-হাজার, একই Process-এ
প্রবেশাধিকার	DOM, `localStorage`	File, Database, Secret, Network
একটি ধীর কাজের প্রভাব	একটি Tab জমে যায়	সব ব্যবহারকারী অপেক্ষা করেন
ভুলের প্রভাব	একজন ব্যবহারকারী	সব তথ্য, সব ব্যবহারকারী

দ্বিতীয় সারিটি সবচেয়ে গুরুত্বপূর্ণ: Server-এর প্রতিটি অনুরোধ বাইরের পৃথিবী থেকে আসে, তাই প্রতিটি Field যাচাই, প্রতিটি অনুমতি পরীক্ষা—Browser কী পাঠানোর "কথা", তা নয়, আসলে কী পাঠিয়েছে।

২৫.৩ REST: Resource, Method ও Status Code

REST (Representational State Transfer) একটি নকশার ধরন: API কতগুলো **Resource** (Lead, Tour, Customer) প্রকাশ করে, প্রতিটির একটি URL, এবং HTTP Method বলে কী করা হচ্ছে।

কাজ	Method ও URL	সফল Status	বৈশিষ্ট্য
তালিকা	`GET /v1/leads?status=new&limit=20`	200	Safe, Idempotent
একটি পড়া	`GET /v1/leads/L-00001`	200	Safe, Idempotent
তৈরি	`POST /v1/leads`	201, সঙ্গে `Location` Header	Idempotent নয়
আংশিক হালনাগাদ	`PATCH /v1/leads/L-00001`	200	
মুছে ফেলা	`DELETE /v1/leads/L-00001`	204	Idempotent

Safe মানে Server-এর অবস্থা বদলায় না; **Idempotent** মানে একই অনুরোধ একবার বা দশবার পাঠালে ফল একই। `POST` Idempotent নয়: দুবার পাঠালে দুটি Lead—অনুচ্ছেদ ২৫.৮-এ এর সমাধান। ব্যর্থতার Status-গুলো:

Status	অর্থ	LeadDesk-এ উদাহরণ
400	অনুরোধ ভুল	অবৈধ ফোন নম্বর, ভাঙা JSON
401	পরিচয় নেই	Token নেই বা ভুল

Status	অর্থ	LeadDesk-এ উদাহরণ
403	পরিচয় আছে, অনুমতি নেই	Agent মুছতে চাইছেন
404	নেই (অথবা আপনার জন্য নেই)	অন্য Agent-এর Lead
409	বর্তমান অবস্থার সঙ্গে সংঘাত	`won` থেকে আবার `contacted`
412	শর্ত মেলেনি	পুরোনো Version দিয়ে হালনাগাদ
413	অনুরোধ খুব বড়	১০ KB-র বেশি Body
429	অনেক বেশি অনুরোধ	Rate Limit
500	Server-এর অপ্রত্যাশিত ভুল	Bug; বিস্তারিত শুধু Log-এ

URL-এর শুরুতে `/v1`: API-র সংস্করণ। পরে কোনো Field-এর অর্থ বদলাতে হলে `/v2` চালু করা যায়, পুরোনো Client না ভেঙে।

২৫.8 Express: Route ও Middleware

`node:http`-এ প্রতিটি পথ ও Method নিজে মেলাতে হয় (অধ্যায় ২০-এর `route` Function)। **Express** এই কাজ সহজ করে: Route, URL-এর Parameter, Query, JSON Body এবং—সবচেয়ে গুরুত্বপূর্ণ—**Middleware**।

> **নোট:** Express একটি npm Package (`npm install express`)। এই বইয়ের QA-তে কোনো Package Install করা হয়নি; এই কম্পিউটারে আগে থেকে Install করা একটি Global Tool-এর ভেতরে Express 4.22.1 পাওয়া গেছে, এবং এই অধ্যায়ের সব Code ও Test সেটি দিয়ে চালানো। আপনার Project-এ `npm install` দিয়ে নিজস্ব Copy নিন।

express-basics.mjs · JavaScript — চলবে: Node.js

```
import express from 'express';

const app = express();
app.use(express.json());
const tours = [
  { id: 1, title: 'Sajek', days: 2 },
  { id: 2, title: 'Kuakata', days: 3 },
];

app.get('/tours', (req, res) => {
  const maxDays = Number(req.query.maxDays ?? Infinity);
  res.json(tours.filter((tour) => tour.days <= maxDays));
});
app.get('/tours/:id', (req, res) => {
  const tour = tours.find((t) => t.id === Number(req.params.id));
  if (!tour) {
    res.status(404).json({ error: 'Tour নেই' });
```

```
    return;
  }
  res.json(tour);
});
app.post('/tours', (req, res) => res.status(201).json({ received: req.body }));

const server = app.listen(0, '127.0.0.1');
await new Promise((resolve) => server.once('listening', resolve));
const base = `http://127.0.0.1:${server.address().port}`;
for (const path of ['/tours?maxDays=2', '/tours/2', '/tours/9']) {
  const response = await fetch(base + path);
  console.log(response.status, JSON.stringify(await response.json()));
}
const posted = await fetch(`${base}/tours`, {
  method: 'POST',
  headers: { 'content-type': 'application/json' },
  body: JSON.stringify({ title: 'Ratargul' }),
});
console.log(posted.status, JSON.stringify(await posted.json()));
server.close();
```

Output

```
200 [{"id":1,"title":"Sajek","days":2}]
200 {"id":2,"title":"Kuakata","days":3}
404 {"error":"Tour নেই"}
201 {"received":{"title":"Ratargul"}}
```

`req.params.id` URL-এর অংশ, `req.query.maxDays` `?`-এর পরের অংশ, `req.body` JSON Body (`express.json()` পড়ে দিয়েছে)। লক্ষ করুন, তিনটিই সব সময় **লেখা** বা অবিশ্বস্ত তথ্য: `Number(...)` দিয়ে রূপান্তর, এবং বাস্তবে যাচাই।

Middleware একটি Function `(req, res, next)`: অনুরোধটি দেখে, কিছু করে, তারপর `next()` ডেকে পরেরটিকে দেয়—অথবা নিজেই উত্তর দিয়ে শৃঙ্খল থামিয়ে দেয়। ক্রমটি `app.use`-এর ক্রম:

middleware-order.mjs · JavaScript — চলবে: Node.js

```
import express from 'express';

const app = express();
let order = [];
const step = (name) => (req, res, next) => {
  order.push(name);
  next();
};

app.use(step('requestId'), step('logger'));
app.use('/v1', (req, res, next) => {
```

```
  order.push('auth');
  if (!req.get('authorization')) {
    res.status(401).json({ error: 'Token নেই' });
    return;
  }
  next();
});
app.get('/v1/leads', (req, res) => {
  order.push('route');
  res.json([]);
});

const server = app.listen(0, '127.0.0.1');
await new Promise((resolve) => server.once('listening', resolve));
const url = `http://127.0.0.1:${server.address().port}/v1/leads`;
for (const headers of [{}, { authorization: 'Bearer demo' }]) {
  order = [];
  const response = await fetch(url, { headers });
  console.log(response.status, order.join(' > '));
}
server.close();
```

Output

```
401 requestId > logger > auth
200 requestId > logger > auth > route
```

Token ছাড়া অনুরোধে `auth` নিজেই 401 দিয়েছে এবং `next()` ডাকেনি, তাই Route পর্যন্ত পৌঁছায়নি—Middleware-এর ক্রমই নিরাপত্তা। একটি ভুল ক্রম (Route আগে, `auth` পরে) হলে সুরক্ষিত তথ্য Token ছাড়াই চলে যেত।

২৫.৫ Error Handling এবং Express 4-এর Async ফাঁদ

Express চারটি Parameter-এর Middleware `(err, req, res, next)`-কে **Error Handler** হিসেবে চেনে: কোনো Route `next(error)` ডাকলে বা Synchronous Error ছুড়লে নিয়ন্ত্রণ সরাসরি সেখানে যায়। কিন্তু Express 4 একটি `async` Function-এর **Rejected Promise** ধরে না—এবং আজকের প্রায় সব Route `async` (Database, Network):

async-errors.mjs · JavaScript — চলবে: Node.js

```
import express from 'express';

process.on('unhandledRejection', (error) => {
  console.log(`unhandledRejection: ${error.message}`);
});

const app = express();
const wrap = (handler) => (req, res, next) =>
```

```
  Promise.resolve(handler(req, res, next)).catch(next);

app.get('/unsafe', async () => {
  throw new Error('Database বন্ধ');
});
app.get('/safe', wrap(async () => {
  throw new Error('Database বন্ধ');
}));
app.use((err, req, res, next) => res.status(500).json({ error: 'Server-এ সমস্যা' }));

const server = app.listen(0, '127.0.0.1');
await new Promise((resolve) => server.once('listening', resolve));
const base = `http://127.0.0.1:${server.address().port}`;
try {
  await fetch(`${base}/unsafe`, { signal: AbortSignal.timeout(500) });
} catch {
  console.log('/unsafe: ৫০০ ms-এ কোনো উত্তর আসেনি');
}
const safe = await fetch(`${base}/safe`);
console.log(`/safe: ${safe.status} ${JSON.stringify(await safe.json())}`);
server.closeAllConnections();
server.close();
```

Output

```
unhandledRejection: Database বন্ধ
/unsafe: ৫০০ ms-এ কোনো উত্তর আসেনি
/safe: 500 {"error":"Server-এ সমস্যা"}
```

`/unsafe`-এর Error কোনো Handler পায়নি: Client কোনো উত্তর পাননি (Timeout পর্যন্ত ঝুলে ছিলেন), এবং Process-এ একটি `unhandledRejection` ঘটেছে—Node.js-এর Default আচরণে এটি পুরো Server বন্ধ করে দেয়; এখানে শুধু প্রদর্শনের জন্য Listener দিয়ে ধরা হয়েছে। `wrap` Rejection ধরে `next(error)`-এ পাঠায়, এবং Error Handler একটি পরিষ্কার 500 দেয়। **Express 5** এই আচরণ নিজেই দেয় (Rejected Promise স্বয়ংক্রিয়ভাবে `next`-এ যায়); Express 4-এ `wrap` বাধ্যতামূলক।

LeadDesk-এর Error Handler সব Error-কে একই আকার দেয়—`{ error: { code, message, details, requestId } }`—যাতে Client একটি নিয়মেই সব ব্যর্থতা সামলাতে পারে। প্রত্যাশিত ব্যর্থতা (`AppError`: Validation, অনুমতি, সংঘাত) নিজের Status নিয়ে যায়; অপ্রত্যাশিত Error-এর বিস্তারিত শুধু Log-এ, Client পান সাধারণ 500 (অধ্যায় ২০)।

২৫.৬ Layered Architecture: Route, Service, Repository

অধ্যায় ২১-এর স্তরগুলো Backend-এ এভাবে বসে:

স্তর	LeadDesk-এ	দায়িত্ব	জানে না
Route (HTTP)	`routes.mjs`	URL, Header, Status Code, JSON	ব্যবসার নিয়ম, Database
Service	`service.mjs`	নিয়ম, অনুমতি, Idempotency, Version	HTTP, Express
Domain	`leads.mjs`	Lead-এর যাচাই ও অবস্থার পরিবর্তন	HTTP, সংরক্ষণ
Repository	`repository.mjs`	সংরক্ষণ ও খোঁজা	HTTP, অনুমতি

Route পাতলা: শুধু HTTP-এর ভাষাকে Service-এর ভাষায় রূপান্তর (`If-Match` Header থেকে `expectedVersion`)। ফলে Service একটি সাধারণ JavaScript Object, যাকে Express ছাড়াই Test করা যায়, এবং একই Service ভবিষ্যতে একটি Queue Worker বা Command-line Tool-ও ব্যবহার করতে পারে।

২৫.৭ Authentication ও Authorization: Token ও Role

Browser-এর জন্য অধ্যায় ২০-এ HttpOnly Cookie ব্যবহার করেছিলেন। অন্য Server, Mobile App বা Script থেকে আসা API অনুরোধে সাধারণত **Bearer Token**: `Authorization: Bearer <token>` Header। LeadDesk-এ নিয়মগুলো:

- Token Environment থেকে আসে, Code-এ নয়; Memory-তে থাকে শুধু Token-এর SHA-256 Hash। খোঁজা হয় Hash দিয়ে, তাই আসল Token Log বা Memory Dump-এ ফাঁস হয় না। (দীর্ঘ এলোমেলো Machine Token-এর জন্য দ্রুত Hash যথেষ্ট; মানুষের Password-এর জন্য scrypt, অধ্যায় ২০।)
- **Role**: `agent` বা `manager`।
- **Permission Matrix**, এক জায়গায় (`service.mjs`):

কাজ	Agent	Manager
Lead তৈরি	হ্যাঁ (নিজের নামে)	হ্যাঁ (কারও নামে নয়)
দেখা	নিজের ও কারও-না-দেওয়া	সব
অবস্থা বদলানো	শুধু নিজের	সব
অন্যকে দেওয়া (`assignee`)	না	হ্যাঁ
মুছে ফেলা	না	হ্যাঁ

অন্য Agent-এর Lead দেখতে চাইলে 404 (অস্তিত্বও লুকানো), দেখা যায় কিন্তু কাজের অনুমতি নেই এমন ক্ষেত্রে 403। Production-এ সাধারণত Token আসে একটি Identity Provider থেকে (OAuth 2.0, OpenID Connect), প্রায়ই JWT আকারে, মেয়াদসহ; Role-এর তথ্য তার ভেতরে। এই Demo-র স্থির Token শুধু শেখার জন্য।

২৫.৮ নির্ভরযোগ্যতা: Pagination, Idempotency, Concurrency

Cursor Pagination: `?page=5` (Offset) সরল, কিন্তু পড়ার মাঝে নতুন Lead যোগ হলে একই Lead দুবার আসে বা বাদ পড়ে, এবং বড় Offset Database-এ ধীর। **Cursor** বলে "এই ID-র পর থেকে": `GET /v1/leads?limit=20&after=L-00020`। Response-এ `next` থাকে পরের পাতার Cursor, শেষ পাতায় `null`।

Idempotency-Key: Mobile Network-এ একটি `POST` পাঠানো হলো, Server Lead তৈরিও করল, কিন্তু উত্তর পৌঁছানোর আগে সংযোগ কেটে গেল। App আবার পাঠাল—এখন দুটি Lead, এবং গ্রাহক দুজন Agent-এর কাছ থেকে দুবার ফোন পান। সমাধান: Client প্রতিটি নতুন কাজের জন্য একটি এলোমেলো Key তৈরি করে `Idempotency-Key` Header-এ পাঠায়, এবং Retry-তে **একই** Key। Server প্রথমবার Key ও ফল মনে রাখে; একই Key আবার এলে নতুন কিছু না করে আগের ফল ফেরত দেয় (Status 200)। একই Key-তে ভিন্ন Body এলে তা Client-এর Bug—409।

Optimistic Concurrency: দুজন একই Lead একসঙ্গে খুলেছেন। Rafi অবস্থা বদলে `contacted` করলেন, তারপর Manager তাঁর পুরোনো পর্দা থেকে `lost` পাঠালেন—Rafi-র পরিবর্তন নীরবে হারিয়ে যেত (**Lost Update**)। প্রতিটি Lead-এর একটি `version`; পড়ার সময় Server `ETag: "v2"` পাঠায়, হালনাগাদে Client `If-Match: "v2"` পাঠায়। Version না মিললে 412: "অন্য কেউ আগে বদলেছে, আবার পড়ে নিন"। "Optimistic" কারণ কোনো Lock নেই—সংঘাত বিরল ধরে নিয়ে শুধু হালনাগাদের মুহূর্তে যাচাই।

২৫.৯ Observability এবং Production-এর অভ্যাস

- **Request ID:** প্রতিটি অনুরোধের একটি ID (`x-request-id`), Response Header-এ, প্রতিটি Log-এ এবং প্রতিটি Error Body-তে। গ্রাহক Error-এর ID জানালে Log-এ ঠিক সেই অনুরোধ খুঁজে পাওয়া যায়।
- **Structured Log:** একটি অনুরোধ, একটি JSON লাইন: Method, Path, Status, সময়, ব্যবহারকারী। কখনো Header (Token) বা Body (ফোন নম্বর) নয়।
- **Health ও Readiness:** `/health` বলে Process জীবিত (Liveness); `/ready` বলে নতুন অনুরোধ নিতে প্রস্তুত (বন্ধ হওয়ার সময় 503)। Load Balancer ও Kubernetes-এর মতো ব্যবস্থা এগুলো দেখে Traffic পাঠায়।
- **Rate Limit:** প্রতি ব্যবহারকারী মিনিটে সর্বোচ্চ অনুরোধ, ছাড়ালে 429 ও `Retry-After`। (এখানে Memory-তে; একাধিক Server-এ Redis-এর মতো ভাগ করা Store লাগে।)
- **Configuration:** সব Setting Environment থেকে, শুরুতেই যাচাই, ভুল হলে স্পষ্ট বার্তায় থামা।
- **Graceful Shutdown:** Deploy বা Restart-এর সময় `SIGTERM` এলে: `ready` বন্ধ (নতুন Traffic আসা থামে), নতুন সংযোগ নেওয়া বন্ধ, চলমান অনুরোধ শেষ হতে দেওয়া, একটি সময়সীমার পর বাকি সংযোগ বন্ধ। নইলে প্রতিটি Deploy-এ কিছু ব্যবহারকারী আধা-সম্পূর্ণ অনুরোধ পান।
- **শুধু `127.0.0.1`:** Demo বাইরে থেকে পৌঁছানো যায় না। Production-এ Server সাধারণত একটি Reverse Proxy-র (Nginx, Cloud Load Balancer) পেছনে, যা HTTPS ও সংযোগের সীমা সামলায়।

২৫.১০ Database: Repository-র পেছনে

LeadDesk-এর Repository Memory-তে, যাতে কোনো Database ছাড়াই Project চলে ও Test হয়। কিন্তু প্রতিটি Method ইচ্ছা করে `async` এবং একটি ছোট চুক্তি মানে (`nextId` , `insert` , `get` , `replace` , `remove` , `list`) —অধ্যায় ২১-এর Repository Pattern। একটি PostgreSQL Adapter একই চুক্তি মানবে: `insert` হবে Parameterized `INSERT` , `replace(lead, expectedVersion)` হবে `UPDATE leads SET ... WHERE id = $1 AND version = $2` এবং বদলানো সারির সংখ্যা ০ হলে `false` —Optimistic Concurrency তখন Database নিজেই নিশ্চিত করে। Idempotency-র তথ্যও তখন Memory-র `Map` -এ নয়, একটি Table-এ, একটি Unique Constraint-সহ, যাতে একাধিক Server-এও কাজ করে। Service-এর একটি লাইনও বদলাতে হবে না। Database-এর বিস্তারিত—Schema, Constraint, Transaction, Index, Migration—পরের অধ্যায়ের বিষয়।

২৫.১১ Project: LeadDesk API

LeadDesk API একটি ভ্রমণ সংস্থার গ্রাহক-জিজ্ঞাসা (Lead) ব্যবস্থাপনার REST API, Express দিয়ে। এতে অনুচ্ছেদ ২৫.৩ থেকে ২৫.৯-এর প্রতিটি কৌশল প্রয়োগ ও পরীক্ষা করা হয়েছে।

> **সতর্কতা:** LeadDesk একটি **শিক্ষামূলক Demo**, Production-প্রস্তুত নয়। তথ্য ও Idempotency-র রেকর্ড Memory-তে (Restart-এ মুছে যায়); Token স্থির ও মেয়াদহীন; HTTPS নেই; Server শুধু `127.0.0.1` -এ। Production-এর জন্য Database (অধ্যায় ২৬), Identity Provider, HTTPS, ভাগ করা Rate Limit Store এবং স্বাধীন Security Review লাগবে।

File-এর কাঠামো

Folder-এর কাঠামো

```
lead-desk/
├── main.mjs                Server চালু, Demo Token, SIGINT/SIGTERM
├── package.json            express Dependency
├── src/
│   ├── config.mjs          Environment থেকে Configuration, Token-এর Hash
│   ├── errors.mjs          AppError ও Helper
│   ├── leads.mjs           Lead-এর যাচাই ও অবস্থার নিয়ম
│   ├── repository.mjs      Memory Repository (async চুক্তি)
│   ├── service.mjs         নিয়ম, অনুমতি, Idempotency, Version
│   ├── middleware.mjs      Request ID, Log, Authentication, Rate Limit
│   ├── error-handler.mjs   404 ও Error Handler
│   ├── routes.mjs          Express Router
│   ├── app.mjs             Middleware-এর ক্রম
│   └── server.mjs          HTTP Server ও Graceful Shutdown
└── tests/                  api, reliability, shutdown (+ helpers.mjs)
```

src/errors.mjs ও src/leads.mjs

errors.mjs · JavaScript — চলবে: Node.js

```
// সব প্রত্যাশিত ব্যর্থতা একটি Class-এ: HTTP Status, যন্ত্রে-পড়ার code, মানুষের বার্তা।
export class AppError extends Error {
  constructor(status, code, message, details = undefined) {
    super(message);
    this.name = 'AppError';
    this.status = status;
    this.code = code;
    this.details = details;
  }
}

export const badRequest = (message, details) =>
  new AppError(400, 'VALIDATION_FAILED', message, details);
export const unauthorized = () =>
  new AppError(401, 'UNAUTHENTICATED', 'বৈধ Bearer Token প্রয়োজন');
export const forbidden = () =>
  new AppError(403, 'FORBIDDEN', 'এই কাজের অনুমতি নেই');
export const notFound = (what = 'Resource') =>
  new AppError(404, 'NOT_FOUND', `${what} পাওয়া যায়নি`);
export const conflict = (code, message) => new AppError(409, code, message);
export const preconditionFailed = () =>
  new AppError(412, 'VERSION_MISMATCH', 'অন্য কেউ আগে বদলেছে; আবার পড়ে নিন');
```

leads.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) — চলবে: Node.js

```
import { badRequest, conflict } from './errors.mjs';

// Lead: ভ্রমণ সংস্থার কাছে একজন সম্ভাব্য গ্রাহকের জিজ্ঞাসা।
export const STATUSES = ['new', 'contacted', 'quoted', 'won', 'lost'];
const NEXT = {
  new: ['contacted', 'lost'],
  contacted: ['quoted', 'lost'],
  quoted: ['won', 'lost'],
  won: [],
  lost: [],
};
const PHONE = /^01[3-9]\d{8}$/;

function check(problems, ok, field, message) {
  if (!ok) {
    problems.push({ field, message });
  }
}

// ...
```

leads.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

export function validateNewLead(body) {
  const b = typeof body === 'object' && body !== null ? body : {};
  const problems = [];
  const text = (v, max) =>
    typeof v === 'string' && v.trim().length > 0 && v.length <= max;
  check(problems, text(b.name, 80), 'name', 'নাম ১-৮০ অক্ষর');
  check(problems, typeof b.phone === 'string' && PHONE.test(b.phone), 'phone',
    '১১ অঙ্কের বাংলাদেশি মোবাইল নম্বর');
  check(problems, text(b.destination, 60), 'destination', 'গন্তব্য প্রয়োজন');
  check(problems, Number.isInteger(b.travelers) && b.travelers >= 1
    && b.travelers <= 50, 'travelers', 'যাত্রী ১ থেকে ৫০ জন');
  if (problems.length > 0) {
    throw badRequest('ইনপুট যাচাই ব্যর্থ', problems);
  }
  return {
    name: b.name.trim(), phone: b.phone, destination: b.destination.trim(),
    travelers: b.travelers,
  };
}

export function assertTransition(from, to) {
  if (!STATUSES.includes(to)) {
    throw badRequest(`অজানা অবস্থা: ${to}`);
  }
  if (!NEXT[from].includes(to)) {
    throw conflict('INVALID_TRANSITION', `${from} থেকে ${to} যাওয়া যায় না`);
  }
}
```

`validateNewLead` শুধু চারটি পরিচিত Field নেয় এবং নতুন Object তৈরি করে (Allowlist)—Body-তে `status: 'won'` বা `assignee` থাকলেও তা উপেক্ষিত। সব ভুল একসঙ্গে, Field-এর নামসহ (`details`)।

src/config.mjs ও src/repository.mjs

config.mjs · JavaScript চলবে: Node.js

```
import { createHash, randomBytes } from 'node:crypto';

export const hashToken = (token) =>
  createHash('sha256').update(token).digest('hex');

// Configuration আসে Environment থেকে; ভুল হলে শুরুতেই স্পষ্ট বার্তায় থামে।
// API_TOKENS-এর রূপ: "id:role:token,id:role:token" (role = agent বা manager)।
export function loadConfig(env = process.env) {
  const port = Number(env.PORT ?? 3025);
```

```
  if (!Number.isInteger(port) || port < 0 || port > 65535) {
    throw new Error(`PORT অবৈধ: ${env.PORT}`);
  }
  const users = new Map();
  for (const entry of (env.API_TOKENS ?? '').split(',').filter(Boolean)) {
    const [id, role, token] = entry.split(':');
    const validRole = ['agent', 'manager'].includes(role);
    if (!id || !validRole || !token || token.length < 24) {
      throw new Error(`API_TOKENS-এ অবৈধ Entry (id "${id}")`);
    }
    // Token নিজে Memory-তে নয়; শুধু তার Hash।
    users.set(hashToken(token), { id, role });
  }
  return {
    port,
    host: '127.0.0.1',
    users,
    rateLimit: { max: Number(env.RATE_LIMIT ?? 60), windowMs: 60_000 },
    shutdownGraceMs: Number(env.SHUTDOWN_GRACE_MS ?? 5_000),
  };
}

// Demo-র জন্য প্রতি চালানোয় নতুন এলোমেলো Token (Code-এ লেখা Token নয়)।
export function demoTokens() {
  const token = () => randomBytes(24).toString('base64url');
  return { manager: token(), rafi: token(), mina: token() };
}
```

repository.mjs · JavaScript — চলবে: Node.js

```
// Memory-ভিত্তিক Repository; প্রতিটি Method async, যাতে পরে Database Adapter
// (অধ্যায় ২৬) একই চুক্তিতে বসানো যায়। সব তথ্য Copy হিসেবে দেওয়া-নেওয়া।
export function createLeadRepository() {
  const leads = new Map();
  let sequence = 0;
  const copy = (lead) => structuredClone(lead);

  return {
    async nextId() {
      sequence += 1;
      return `L-${String(sequence).padStart(5, '0')}`;
    },
    async insert(lead) {
      leads.set(lead.id, copy(lead));
    },
    async get(id) {
      return leads.has(id) ? copy(leads.get(id)) : null;
    },
    // শর্তসাপেক্ষ হালনাগাদ: সংরক্ষিত version মিললে তবেই (Optimistic Concurrency)।
```

```
    async replace(lead, expectedVersion) {
      if (leads.get(lead.id)?.version !== expectedVersion) {
        return false;
      }
      leads.set(lead.id, copy(lead));
      return true;
    },
    async remove(id) {
      return leads.delete(id);
    },
    // Cursor Pagination: id-র ক্রমে, after-এর পরের limit-টি, এবং আরও আছে কি না।
    async list({ where = () => true, after = '', limit = 20 }) {
      const rows = [...leads.values()]
        .filter((lead) => lead.id > after && where(lead))
        .slice(0, limit + 1);
      const items = rows.slice(0, limit).map(copy);
      return { items, hasMore: rows.length > limit };
    },
  };
}
```

src/service.mjs

service.mjs · JavaScript (অংশ ১ / ৩ (Part 1 of 3)) চলবে: Node.js

```
import { validateNewLead, assertTransition, STATUSES } from './leads.mjs';
import {
  badRequest, forbidden, notFound, conflict, preconditionFailed,
} from './errors.mjs';

// অনুমতির নিয়ম এক জায়গায়। Agent দেখেন নিজের ও কারও-না-দেওয়া Lead;
// বদলাতে পারেন শুধু নিজেরটি। Manager সব।
const canSee = (user, lead) => user.role === 'manager'
  || lead.assignee === user.id || lead.assignee === null;
const canEdit = (user, lead) =>
  user.role === 'manager' || lead.assignee === user.id;

export function createLeadService({ repository, now = () => new Date() }) {
  const idempotency = new Map();

  async function create(user, body, idempotencyKey) {
    const input = validateNewLead(body);
    const key = idempotencyKey ? `${user.id}:${idempotencyKey}` : null;
    const fingerprint = JSON.stringify(input);
    const previous = key ? idempotency.get(key) : undefined;
    if (previous) {
      if (previous.fingerprint !== fingerprint) {
        throw conflict('IDEMPOTENCY_MISMATCH', 'একই Key, ভিন্ন অনুরোধ');
```

```
      }
      return { lead: await repository.get(previous.leadId), replayed: true };
    }
    const lead = {
      id: await repository.nextId(), ...input, status: 'new',
      assignee: user.role === 'agent' ? user.id : null,
      version: 1, createdAt: now().toISOString(),
    };
    await repository.insert(lead);
    if (key) {
      idempotency.set(key, { fingerprint, leadId: lead.id });
    }
    return { lead, replayed: false };
  }

// ...
```

service.mjs · JavaScript (অংশ ২ / ৩ (Part 2 of 3)) চলবে: Node.js

```
// ...

  async function get(user, id) {
    const lead = await repository.get(id);
    if (!lead || !canSee(user, lead)) {
      throw notFound('Lead');
    }
    return lead;
  }

  async function list(user, { status, after = '', limit = '20' }) {
    const size = Number(limit);
    if (!Number.isInteger(size) || size < 1 || size > 100) {
      throw badRequest('limit ১ থেকে ১০০');
    }
    if (status !== undefined && !STATUSES.includes(status)) {
      throw badRequest(`অজানা status: ${status}`);
    }
    const where = (lead) =>
      canSee(user, lead) && (!status || lead.status === status);
    return repository.list({ where, after: String(after), limit: size });
  }

// ...
```

service.mjs · JavaScript (অংশ ৩ / ৩ (Part 3 of 3)) চলবে: Node.js

```
// ...

  async function update(user, id, patch, expectedVersion) {
```

```
    const lead = await get(user, id);
    if (!canEdit(user, lead)) {
      throw forbidden();
    }
    if (expectedVersion !== undefined && expectedVersion !== lead.version) {
      throw preconditionFailed();
    }
    const fields = Object.keys(patch ?? {});
    const unknown = fields.filter((k) => !['status', 'assignee'].includes(k));
    if (unknown.length > 0 || fields.length === 0) {
      throw badRequest('শুধু status বা assignee বদলানো যায়', unknown);
    }
    const next = { ...lead, version: lead.version + 1 };
    if ('status' in patch) {
      assertTransition(lead.status, patch.status);
      next.status = patch.status;
    }
    if ('assignee' in patch) {
      if (user.role !== 'manager') {
        throw forbidden();
      }
      const { assignee } = patch;
      next.assignee = typeof assignee === 'string' ? assignee : null;
    }
    if (!(await repository.replace(next, lead.version))) {
      throw preconditionFailed();
    }
    return next;
  }

  async function remove(user, id) {
    await get(user, id);
    if (user.role !== 'manager') {
      throw forbidden();
    }
    await repository.remove(id);
  }

  return { create, get, list, update, remove };
}
```

অনুমতির নিয়ম দুটি ছোট Function-এ (`canSee`, `canEdit`), এবং প্রতিটি কাজ সেগুলোর মধ্য দিয়ে যায়: তালিকার `where`-এও `canSee`, যাতে তালিকা দিয়েও অন্যের Lead বেরিয়ে না আসে। `update`-এ তিনটি স্তরের যাচাই: অনুমতি, `If-Match`-এর Version, এবং অবস্থার পরিবর্তনের নিয়ম; শেষে Repository-র শর্তসাপেক্ষ `replace`, যা দুটি একসঙ্গে আসা হালনাগাদের মধ্যে দ্বিতীয়টিকে ধরে।

src/middleware.mjs ও src/error-handler.mjs

middleware.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) চলবে: Node.js

```
import { randomUUID } from 'node:crypto';
import { AppError, unauthorized } from './errors.mjs';
import { hashToken } from './config.mjs';

// প্রতিটি অনুরোধের একটি ID: Response Header-এ ও প্রতিটি Log-এ।
export function requestId(req, res, next) {
  const incoming = req.get('x-request-id');
  req.id = incoming && /^[\w-]{1,64}$/.test(incoming) ? incoming : randomUUID();
  res.set('x-request-id', req.id);
  next();
}

// Structured Log: একটি অনুরোধ, একটি JSON লাইন। Header বা Body Log হয় না।
export const requestLogger = (log) => (req, res, next) => {
  const started = process.hrtime.bigint();
  res.on('finish', () => log({
    requestId: req.id, method: req.method, path: req.path,
    status: res.statusCode,
    user: req.user?.id ?? null,
    ms: Number(process.hrtime.bigint() - started) / 1e6,
  }));
  next();
};

// ...
```

middleware.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

export const authenticate = (users) => (req, res, next) => {
  const [scheme, token] = (req.get('authorization') ?? '').split(' ');
  const valid = scheme === 'Bearer' && Boolean(token);
  const user = valid ? users.get(hashToken(token)) : undefined;
  if (!user) {
    next(unauthorized());
    return;
  }
  req.user = user;
  next();
};

// প্রতি ব্যবহারকারী নির্দিষ্ট সময়-জানালায় সর্বোচ্চ max অনুরোধ।
export function rateLimit({ max, windowMs, now = Date.now }) {
  const windows = new Map();
  return (req, res, next) => {
```

```
    const key = req.user?.id ?? req.ip;
    const current = windows.get(key);
    const window = current && now() - current.start < windowMs
      ? current : { start: now(), count: 0 };
    window.count += 1;
    windows.set(key, window);
    if (window.count > max) {
      const retry = Math.ceil((window.start + windowMs - now()) / 1000);
      res.set('retry-after', String(retry));
      next(new AppError(429, 'RATE_LIMITED', 'অনেক অনুরোধ; কিছুক্ষণ পরে চেষ্টা করুন'));
      return;
    }
    next();
  };
}
```

error-handler.mjs · JavaScript — চলবে: Node.js

```
import { AppError, notFound } from './errors.mjs';

export const notFoundHandler = (req, res, next) => next(notFound('পথ'));

// Express চারটি Parameter-এর Middleware-কে Error Handler হিসেবে চেনে।
// সব Error একই আকারে: { error: { code, message, details?, requestId } }।
export const errorHandler = (log) => (err, req, res, next) => {
  let error = err;
  if (err.type === 'entity.parse.failed') {
    error = new AppError(400, 'INVALID_JSON', 'JSON পড়া যায়নি');
  } else if (err.type === 'entity.too.large') {
    error = new AppError(413, 'PAYLOAD_TOO_LARGE', 'অনুরোধ খুব বড়');
  } else if (!(err instanceof AppError)) {
    // অপ্রত্যাশিত: বিস্তারিত শুধু Server Log-এ, ব্যবহারকারী পান সাধারণ বার্তা।
    log({ requestId: req.id, level: 'error', message: err.message });
    error = new AppError(500, 'INTERNAL_ERROR', 'Server-এ সমস্যা হয়েছে');
  }
  if (res.headersSent) {
    next(err);
    return;
  }
  const { status, code, message, details } = error;
  const body = { code, message, details, requestId: req.id };
  res.status(status).json({ error: body });
};
```

Client চাইলে নিজের `x-request-id` পাঠাতে পারেন (যেমন Frontend-এর একটি ID), কিন্তু শুধু নিরাপদ অক্ষরে ও ৬৪ অক্ষর পর্যন্ত—নইলে Log-এ ক্ষতিকর লেখা ঢুকতে পারত। `express.json()`-এর দুটি নিজস্ব Error (`entity.parse.failed`, `entity.too.large`) Error Handler-এ নিজস্ব Code পায়।

src/routes.mjs, src/app.mjs ও src/server.mjs

routes.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) — চলবে: Node.js

```
import express from 'express';

// Express 4 async Function-এর Rejected Promise নিজে ধরে না; এই Wrapper ধরে
// next(error)-এ পাঠায় (Express 5-এ এটি অন্তর্নির্মিত)।
const wrap = (handler) => (req, res, next) =>
  Promise.resolve(handler(req, res, next)).catch(next);

const etag = (lead) => `"v${lead.version}"`;
const versionFrom = (header) => {
  const match = /^"v(\d+)"$/.exec(header ?? '');
  return match ? Number(match[1]) : undefined;
};

export function leadRoutes(service) {
  const router = express.Router();

  router.post('/', wrap(async (req, res) => {
    const key = req.get('idempotency-key');
    const { lead, replayed } = await service.create(req.user, req.body, key);
    res.status(replayed ? 200 : 201)
      .location(`/v1/leads/${lead.id}`).set('etag', etag(lead)).json(lead);
  }));

// ...
```

routes.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) — চলবে: Node.js

```
// ...

  router.get('/', wrap(async (req, res) => {
    const page = await service.list(req.user, req.query);
    const last = page.items.at(-1);
    res.json({ ...page, next: page.hasMore && last ? last.id : null });
  }));

  router.get('/:id', wrap(async (req, res) => {
    const lead = await service.get(req.user, req.params.id);
    res.set('etag', etag(lead)).json(lead);
  }));

  router.patch('/:id', wrap(async (req, res) => {
    const expected = versionFrom(req.get('if-match'));
    const { user, params, body } = req;
    const lead = await service.update(user, params.id, body, expected);
    res.set('etag', etag(lead)).json(lead);
  }));
```

```
  router.delete('/:id', wrap(async (req, res) => {
    await service.remove(req.user, req.params.id);
    res.status(204).end();
  }));

  return router;
}
```

app.mjs · JavaScript — চলবে: Node.js

```
import express from 'express';
import {
  requestId, requestLogger, authenticate, rateLimit,
} from './middleware.mjs';
import { notFoundHandler, errorHandler } from './error-handler.mjs';
import { leadRoutes } from './routes.mjs';

// Middleware-এর ক্রমই নিরাপত্তা: ID ও Log সবার আগে, তারপর Body-র সীমা,
// তারপর Health (Token ছাড়া), তারপর Authentication, সীমা, Route, শেষে Error।
export function createApp(deps) {
  const { config, service, log = () => {}, state = { ready: true } } = deps;
  const app = express();
  app.disable('x-powered-by');
  app.use(requestId);
  app.use(requestLogger(log));
  app.use(express.json({ limit: '10kb' }));

  app.get('/health', (req, res) => res.json({ status: 'ok' }));
  app.get('/ready', (req, res) => {
    res.status(state.ready ? 200 : 503).json({ ready: state.ready });
  });

  app.use('/v1', authenticate(config.users), rateLimit(config.rateLimit));
  app.use('/v1/leads', leadRoutes(service));

  app.use(notFoundHandler);
  app.use(errorHandler(log));
  return app;
}
```

server.mjs · JavaScript — চলবে: Node.js

```
import { createServer } from 'node:http';
import { createApp } from './app.mjs';
import { createLeadService } from './service.mjs';
import { createLeadRepository } from './repository.mjs';

// সব অংশ জোড়া লাগিয়ে Server চালু; shutdown() নতুন সংযোগ বন্ধ করে,
```

```
// চলমান অনুরোধ শেষ হতে দেয়, এবং সময়সীমার পরে বাকি সংযোগ কেটে দেয়।
export async function startServer(config, { log = () => {}, service } = {}) {
  const state = { ready: true };
  const leads = service
    ?? createLeadService({ repository: createLeadRepository() });
  const app = createApp({ config, service: leads, log, state });
  const server = createServer(app);
  await new Promise((resolve) => {
    server.listen(config.port, config.host, resolve);
  });

  let closing = null;
  function shutdown() {
    closing ??= new Promise((resolve) => {
      state.ready = false;
      server.close(resolve);
      server.closeIdleConnections();
      const force = () => server.closeAllConnections();
      setTimeout(force, config.shutdownGraceMs).unref();
    });
    return closing;
  }

  const { port } = server.address();
  return { url: `http://${config.host}:${port}`, shutdown, state };
}
```

`createApp`-এর ক্রম পড়ুন যেন একটি অনুরোধ ওপর থেকে নিচে যাচ্ছে: ID, Log, Body-র সীমা, Token-ছাড়া Health, তারপর `/v1`-এর সব কিছুর আগে Authentication ও Rate Limit, তারপর Route, এবং যা মেলেনি তার জন্য 404, শেষে Error Handler। `shutdown()` একবারই চলে (`closing ??=`), Node.js-এর `server.close` (নতুন সংযোগ বন্ধ, চলমানগুলো শেষ হলে Callback), `closeIdleConnections` (অপেক্ষারত Keep-alive সংযোগ বন্ধ) এবং সময়সীমার পরে `closeAllConnections`।

main.mjs

main.mjs · JavaScript — চলবে: Node.js

```
import { loadConfig, demoTokens } from './src/config.mjs';
import { startServer } from './src/server.mjs';

// API_TOKENS না দিলে Demo-র জন্য প্রতি চালানোয় নতুন Token (শুধু এই Terminal-এ দেখায়)।
let env = process.env;
if (!env.API_TOKENS) {
  const t = demoTokens();
  const tokens = [`manager:manager:${t.manager}`, `rafi:agent:${t.rafi}`,
    `mina:agent:${t.mina}`];
  env = { ...env, API_TOKENS: tokens.join(',') };
  console.log(`Demo Token (manager): ${t.manager}`);
```

```
  console.log(`Demo Token (rafi):    ${t.rafi}`);
  console.log(`Demo Token (mina):    ${t.mina}`);
}

const config = loadConfig(env);
const log = (entry) => console.log(JSON.stringify(entry));
const server = await startServer(config, { log });
console.log(`LeadDesk API (শিক্ষামূলক Demo, শুধু এই কম্পিউটারে): ${server.url}`);

for (const signal of ['SIGINT', 'SIGTERM']) {
  process.once(signal, async () => {
    console.log(`${signal}: নতুন অনুরোধ বন্ধ, চলমান অনুরোধ শেষ হওয়ার অপেক্ষা...`);
    await server.shutdown();
    console.log('বন্ধ হয়েছে।');
    process.exit(0);
  });
}
```

উদাহরণ অনুরোধ

`node main.mjs` চালালে Terminal-এ তিনটি Demo Token দেখায়। নিচের Command-গুলো উদাহরণ (Windows PowerShell-এ `curl.exe`); `<TOKEN>`-এর জায়গায় একটি Token বসান। এই বইয়ের QA-তে এগুলো হাতে চালানো হয়নি; একই অনুরোধগুলো স্বয়ংক্রিয় Test-এ যাচাই করা হয়েছে।

Terminal

```
curl -i -X POST http://127.0.0.1:3025/v1/leads \
  -H "Authorization: Bearer <TOKEN>" -H "Content-Type: application/json" \
  -H "Idempotency-Key: form-7f3a" \
  -d '{"name":"Karim Uddin","phone":"01712345678",
       "destination":"Sylhet","travelers":4}'
curl -i "http://127.0.0.1:3025/v1/leads?status=new&limit=20" \
  -H "Authorization: Bearer <TOKEN>"
curl -i -X PATCH http://127.0.0.1:3025/v1/leads/L-00001 \
  -H "Authorization: Bearer <TOKEN>" -H "Content-Type: application/json" \
  -H 'If-Match: "v1"' -d '{"status":"contacted"}'
```

Test

api.test.mjs · JavaScript · নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) · চলবে: Node.js

```
// ...

test('Object-level Authorization: অন্য Agent-এর Lead দেখা যায় না', async (t) => {
  const { call } = await startTestServer(t);
  await call('POST', '/v1/leads', { as: 'rafi', body: newLead() });
  await call('POST', '/v1/leads',
    { as: 'manager', body: newLead({ name: 'Unassigned' }) });
```

```
  const other = await call('GET', '/v1/leads/L-00001', { as: 'mina' });
  assert.equal(other.status, 404);
  const minaList = await call('GET', '/v1/leads', { as: 'mina' });
  assert.deepEqual(minaList.body.items.map((l) => l.id), ['L-00002']);
  const assign = await call('PATCH', '/v1/leads/L-00002',
    { as: 'mina', body: { assignee: 'mina' } });
  assert.equal(assign.status, 403);
});

// ...
```

reliability.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

test('Idempotency-Key: একই অনুরোধ দুবার, একটিই Lead', async (t) => {
  const { call } = await startTestServer(t);
  const headers = { 'idempotency-key': 'form-7f3a' };
  const send = () =>
    call('POST', '/v1/leads', { as: 'rafi', body: newLead(), headers });
  const first = await send();
  const retry = await send();
  assert.deepEqual([first.status, retry.status], [201, 200]);
  assert.equal(retry.body.id, first.body.id);
  const list = await call('GET', '/v1/leads', { as: 'rafi' });
  assert.equal(list.body.items.length, 1);
  const changed = await call('POST', '/v1/leads',
    { as: 'rafi', body: newLead({ travelers: 9 }), headers });
  assert.equal(changed.status, 409);
  assert.equal(changed.body.error.code, 'IDEMPOTENCY_MISMATCH');
});

// ...
```

shutdown.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

test('shutdown(): চলমান অনুরোধ সম্পূর্ণ, তারপর বন্ধ', async (t) => {
  let release;
  const gate = new Promise((resolve) => {
    release = resolve;
  });
  // নকল Service: list ধীর, যতক্ষণ না Test "release" করে।
  const service = { list: async () => {
    await gate;
    return { items: [], hasMore: false };
  } };
  const { server } = await startTestServer(t, { service });
```

```
  const auth = { authorization: `Bearer ${TOKENS.rafi}` };

  const inFlight = fetch(`${server.url}/v1/leads`, { headers: auth });
  await new Promise((resolve) => setTimeout(resolve, 100));
  const closed = server.shutdown();
  assert.equal(server.state.ready, false);

  const fresh = { headers: { connection: 'close' } };
  await assert.rejects(fetch(`${server.url}/health`, fresh));
  release();
  const response = await inFlight;
  assert.equal(response.status, 200);
  assert.deepEqual(await response.json(),
    { items: [], hasMore: false, next: null });
  await closed;
});

// ...
```

Shutdown Test-এর কৌশলটি লক্ষ করুন: একটি নকল Service, যার `list` একটি Promise-এর জন্য অপেক্ষা করে। Test একটি অনুরোধ পাঠায় (যা ঝুলে থাকে), তারপর `shutdown()` ডাকে, প্রমাণ করে `ready` বন্ধ এবং নতুন সংযোগ প্রত্যাখ্যাত, তারপর "ছেড়ে দেয়"—এবং পুরোনো অনুরোধটি সম্পূর্ণ উত্তর পায়। আসল সময়ের ওপর নির্ভরতা নেই (অধ্যায় ২২)।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd lead-desk
npm install
npm test
npm start
```

`npm install` Express আনে। `npm test` ১০টি Test চালায়: `api` ৪ (REST-এর পূর্ণ চক্র, প্রতিটি Error Status ও Body-র আকার, Object-level Authorization, Cursor Pagination ও Filter), `reliability` ৪ (Idempotency-Key, `If-Match` ও 412, Rate Limit ও `Retry-After`, Request ID ও Log), এবং `shutdown` ২ (চলমান অনুরোধসহ Graceful Shutdown, Configuration-এর ভুল); শেষে `# pass 10`, `# fail 0`। প্রতিটি Test নিজের Server চালায়, এলোমেলো Port-এ, শুধু `127.0.0.1`-এ। `npm start` Port ৩০২৫-এ চালু করে; Ctrl+C চাপলে Graceful Shutdown-এর বার্তা দেখায়।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`Cannot find package 'express'`	`npm install` হয়নি	Project Folder-এ `npm install`
অনুরোধ ঝুলে থাকে, উত্তর আসে না	`async` Route-এ `wrap` নেই	`wrap` বা Express 5
সব অনুরোধে 401	Token-এর আগে `Bearer` নেই, বা ভুল Token	Header দেখুন
413	Body ১০ KB-র বেশি	ছোট করুন বা সীমা বিবেচনা
`EADDRINUSE`	Port ব্যস্ত	`PORT=3026 npm start`
Ctrl+C-এর পর দেরিতে বন্ধ	চলমান বা Keep-alive সংযোগ	স্বাভাবিক; সর্বোচ্চ `SHUTDOWN_GRACE_MS`

২৫.১২ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
Express 4-এ `async` Route, `wrap` ছাড়া	অনুরোধ ঝুলে থাকে, Process বন্ধ হতে পারে	`wrap` বা Express 5
Route-এ ব্যবসার নিয়ম ও SQL	পরীক্ষা ও পুনর্ব্যবহার কঠিন	Service ও Repository
Middleware-এর ভুল ক্রম	Authentication-এর আগেই Route	ক্রম নিয়ে Test
`req.body`-কে সরাসরি সংরক্ষণ	অনাকাঙ্ক্ষিত Field (`status`, `role`)	Allowlist যাচাই
সব ব্যর্থতায় 500 বা 200	Client বুঝতে পারে না	সঠিক Status ও একই আকারের Error
Offset Pagination বড় তালিকায়	ধীর, ডুপ্লিকেট বা বাদ	Cursor
Retry-তে নতুন Key	ডুপ্লিকেট রেকর্ড	একই `Idempotency-Key`
Version ছাড়া হালনাগাদ	Lost Update	`ETag` ও `If-Match`
Log-এ Header বা Body	Token ও ব্যক্তিগত তথ্য ফাঁস	শুধু নির্বাচিত Field
`process.exit()` সঙ্গে সঙ্গে	চলমান অনুরোধ কেটে যায়	Graceful Shutdown
Code-এ Token বা Password	Git-এ ফাঁস	Environment, Secret Manager

২৫.১৩ Debugging নির্দেশিকা

১. **Request ID দিয়ে শুরু করুন:** Error Body-র `requestId` দিয়ে Log-এ ঠিক সেই অনুরোধের লাইন।
২. `curl -i` : Status ও প্রতিটি Header দেখায় (`ETag` , `Retry-After` , `Location`)।
৩. **ঝুলে থাকা অনুরোধ:** কোনো Middleware `next()` ডাকেনি বা উত্তর দেয়নি, অথবা `async` Error ধরা পড়েনি।
৪. **401 বনাম 403 বনাম 404:** পরিচয়, অনুমতি, না অস্তিত্ব—অনুচ্ছেদ ২৫.৩-এর টেবিল।
৫. `node --inspect main.mjs` : Chrome DevTools-এ Breakpoint দিয়ে Server-এর Code ধাপে ধাপে চালান।
৬. **একটি অংশ আলাদা করুন:** Service-কে সরাসরি ডেকে দেখুন—সমস্যা HTTP-এ, না নিয়মে।

২৫.১৪ পেশাদার অভ্যাস

- **প্রতিটি Route পাতলা,** নিয়ম Service-এ, সংরক্ষণ Repository-তে।
- **প্রতিটি অনুরোধের Body Allowlist দিয়ে যাচাই,** Server-এ, সব সময়।
- **একই আকারের Error,** Request ID-সহ।
- **অনুমতির নিয়ম এক জায়গায়,** এবং প্রতিটি Role ও প্রতিটি কাজের জন্য Test।
- `POST` **-এ Idempotency-Key, হালনাগাদে Version।**
- **Health, Readiness ও Graceful Shutdown প্রথম দিন থেকে।**
- **Configuration Environment থেকে, শুরুতেই যাচাই।**
- **API-র সংস্করণ URL-এ (** `/v1` **),** এবং পরিবর্তন লিখিত (OpenAPI Specification)।

২৫.১৫ অনুশীলনী

প্রাথমিক স্তর

১. Express-এ `GET /health` লিখুন, যা `{ status: 'ok', uptime: <সেকেন্ড> }` দেয়।
২. নিচের প্রতিটির জন্য সঠিক Status লিখুন: (ক) নতুন Lead তৈরি, (খ) Token নেই, (গ) Agent মুছতে চাইছেন, (ঘ) একই Lead-এ পুরোনো Version।
৩. Safe ও Idempotent-এর পার্থক্য উদাহরণসহ লিখুন।
৪. একটি Middleware লিখুন, যা প্রতিটি Response-এ `x-response-time` Header বসায়।
৫. Express 4-এ `async` Route-এ Error কেন বিশেষ যত্ন চায়? দুই বাক্যে।

মধ্যম স্তর

৬. LeadDesk-এ `GET /v1/leads?destination=sylhet` Filter যোগ করুন (ছোট-বড় হাতের পার্থক্য ছাড়া), এবং একটি Test লিখুন।

৭. একটি `requireRole('manager')` Middleware লিখুন এবং একটি Route-এ ব্যবহার করুন।

৮. একটি ৪০০ ms-এর সময়সীমার Middleware লিখুন, যা ধীর Route-এ 503 দেয়।

৯. Cursor Pagination-এ ক্লায়েন্টের দিক থেকে সব পাতা পড়ার একটি Loop লিখুন (LeadDesk-এর বিরুদ্ধে)।

চ্যালেঞ্জ স্তর

১০. Idempotency-র রেকর্ড ২৪ ঘণ্টা পর মুছে যাওয়ার নকশা করুন (ঘড়ি Inject), এবং Test।

১১. Lead-এর জন্য একটি `POST /v1/leads/:id/notes` Sub-resource নকশা করুন: URL, Status, অনুমতি, এবং Service-এর Method।

১২. LeadDesk-এর Repository-র জন্য একটি `node:sqlite` Adapter লিখুন, যা একই চুক্তি মানে, এবং `replace`-এর Optimistic Concurrency SQL-এ প্রয়োগ করুন।

২৫.১৬ অনুশীলনীর সমাধান

সমাধান ১

health.mjs · JavaScript — চলবে: Node.js

```
import express from 'express';

const app = express();
const started = Date.now();
app.get('/health', (req, res) => {
  res.json({ status: 'ok', uptime: Math.floor((Date.now() - started) / 1000) });
});

const server = app.listen(0, '127.0.0.1');
await new Promise((resolve) => server.once('listening', resolve));
const response = await fetch(`http://127.0.0.1:${server.address().port}/health`);
const body = await response.json();
console.log(response.status, body.status, Number.isInteger(body.uptime));
server.close();
```

Output

```
200 ok true
```

সমাধান ২ — (ক) 201, সঙ্গে `Location: /v1/leads/<id>`; (খ) 401; (গ) 403—পরিচয় আছে, Lead দেখাও যায়, কিন্তু মুছে ফেলার অনুমতি নেই; (ঘ) 412 (`If-Match` মেলেনি)।

সমাধান ৩ — Safe: Server-এর অবস্থা বদলায় না (`GET /v1/leads`)। Idempotent: একবার বা বহুবার পাঠালে একই ফল— `DELETE /v1/leads/L-1` দুবার পাঠালেও Lead একবারই মুছে যায় (দ্বিতীয়বার 404, কিন্তু অবস্থা একই)। সব Safe Method Idempotent, কিন্তু সব Idempotent Method Safe নয় (`DELETE`, `PUT`)। `POST` কোনোটিই নয়, তাই `Idempotency-Key` লাগে।

সমাধান ৪

response-time.mjs · JavaScript — চলবে: Node.js

```
import express from 'express';

const responseTime = (req, res, next) => {
  const started = process.hrtime.bigint();
  const original = res.writeHead;
  res.writeHead = function writeHead(...args) {
    const ms = Number(process.hrtime.bigint() - started) / 1e6;
    res.setHeader('x-response-time', `${ms.toFixed(1)}ms`);
    return original.apply(this, args);
  };
  next();
};

const app = express();
app.use(responseTime);
app.get('/', (req, res) => res.json({ ok: true }));

const server = app.listen(0, '127.0.0.1');
await new Promise((resolve) => server.once('listening', resolve));
const response = await fetch(`http://127.0.0.1:${server.address().port}/`);
console.log(/^\d+\.\dms$/.test(response.headers.get('x-response-time')));
server.close();
```

Output

```
true
```

Header Response পাঠানোর ঠিক আগে বসাতে হয়; `finish` Event-এ বসালে দেরি হয়ে যেত, কারণ Header ততক্ষণে চলে গেছে। তাই `writeHead`-কে মুড়ে দেওয়া। সময়ের মান প্রতিবার আলাদা, তাই শুধু আকার যাচাই।

সমাধান ৫ — Express 4 একটি `async` Function-এর Rejected Promise ধরে না, তাই Error Handler-এ পৌঁছায় না: Client উত্তর পান না, এবং `unhandledRejection` Node.js-এর Default আচরণে পুরো Process বন্ধ করে। `wrap` (বা Express 5) Rejection-কে `next(error)`-এ পাঠায়।

সমাধান ৬ — `service.mjs`-এর `list`-এ: `const place = destination?.toLowerCase();` এবং `where`-এ `&& (!place || lead.destination.toLowerCase().includes(place))`। Test (`api.test.mjs`-এর ধাঁচে): দুটি ভিন্ন গন্তব্যের Lead তৈরি, তারপর `GET /v1/leads?destination=SYL` শুধু Sylhet-এরটি দেয়। মনে রাখুন, Query সব সময় লেখা: দৈর্ঘ্যের সীমা দিন (যেমন ৬০), এবং Database-এ গেলে Parameterized `LIKE` (অধ্যায় ২০-এর অনুশীলনী)।

সমাধান ৭

require-role.mjs · JavaScript — চলবে: Node.js

```
import express from 'express';

const requireRole = (role) => (req, res, next) => {
  if (req.user?.role !== role) {
    res.status(403).json({ error: { code: 'FORBIDDEN' } });
    return;
  }
  next();
};

const app = express();
app.use((req, res, next) => {
  req.user = { id: 'x', role: req.get('x-demo-role') };
  next();
});
app.delete('/v1/leads/:id', requireRole('manager'),
  (req, res) => res.status(204).end());

const server = app.listen(0, '127.0.0.1');
await new Promise((resolve) => server.once('listening', resolve));
const url = `http://127.0.0.1:${server.address().port}/v1/leads/L-1`;
for (const role of ['agent', 'manager']) {
  const headers = { 'x-demo-role': role };
  const response = await fetch(url, { method: 'DELETE', headers });
  console.log(role, response.status);
}
server.close();
```

Output

```
agent 403
manager 204
```

এখানে `x-demo-role` Header শুধু প্রদর্শনের জন্য, Authentication-এর বদলে; বাস্তবে Role আসে যাচাই করা Token থেকে, কখনো Client-এর পাঠানো Header থেকে নয়। Route-স্তরের Role যাচাই সহজ ও স্পষ্ট, কিন্তু মালিকানার নিয়ম (Agent শুধু নিজের Lead) Service-এই থাকা উচিত।

সমাধান ৮

timeout.mjs · JavaScript — চলবে: Node.js

```
import express from 'express';

const timeout = (ms) => (req, res, next) => {
  const timer = setTimeout(() => {
    if (!res.headersSent) {
      res.status(503).json({ error: { code: 'TIMEOUT' } });
    }
  }, ms);
  res.on('finish', () => clearTimeout(timer));
  res.on('close', () => clearTimeout(timer));
  next();
};

const app = express();
app.use(timeout(400));
app.get('/fast', (req, res) => res.json({ ok: true }));
app.get('/slow', (req, res) => {
  setTimeout(() => {
    if (!res.headersSent) {
      res.json({ ok: true });
    }
  }, 1_000);
});

const server = app.listen(0, '127.0.0.1');
await new Promise((resolve) => server.once('listening', resolve));
const base = `http://127.0.0.1:${server.address().port}`;
for (const path of ['/fast', '/slow']) {
  console.log(path, (await fetch(base + path)).status);
}
server.closeAllConnections();
server.close();
```

Output

```
/fast 200
/slow 503
```

৪০০ ও ১,০০০ ms-এর পার্থক্য বড়, তাই ফল স্থির। লক্ষ করুন, 503 পাঠানোর পরও ধীর কাজটি পেছনে চলতেই থাকে (`headersSent` যাচাই তাই জরুরি)। আসল কাজ থামাতে `AbortSignal` কাজটির ভেতরে পৌঁছে দিতে হয় (যেমন Database Query বাতিল)।

সমাধান ৯

all-pages.mjs · JavaScript — চলবে: Node.js

```
import { loadConfig } from '../../project/lead-desk/src/config.mjs';
import { startServer } from '../../project/lead-desk/src/server.mjs';

const token = 'reader-token-for-the-pagination-demo';
const config = loadConfig({ PORT: '0', API_TOKENS: `boss:manager:${token}` });
const server = await startServer(config);
const headers = {
  authorization: `Bearer ${token}`, 'content-type': 'application/json',
};
for (let i = 1; i <= 7; i += 1) {
  const body = JSON.stringify({
    name: `Guest ${i}`, phone: '01812345678',
    destination: 'Bandarban', travelers: i,
  });
  await fetch(`${server.url}/v1/leads`, { method: 'POST', headers, body });
}

const ids = [];
let after = '';
let pages = 0;
do {
  const page = await (await fetch(`${server.url}/v1/leads?limit=3&after=${after}`,
    { headers })).json();
  ids.push(...page.items.map((lead) => lead.id));
  after = page.next;
  pages += 1;
} while (after);
console.log(pages, ids.length, ids.at(-1));
await server.shutdown();
```

Output

```
3 7 L-00007
```

Client শুধু `next` অনুসরণ করে; Cursor-এর ভেতরের গঠন তার জানার দরকার নেই, তাই Server পরে Cursor-এর রূপ বদলাতে পারে (যেমন তারিখ ও ID একসঙ্গে)।

সমাধান ১০ — `createLeadService`-এ `now` আগে থেকেই Inject করা; Idempotency-র রেকর্ডে `createdAt: now().getTime()` রাখুন, এবং খোঁজার সময় `now() - previous.createdAt > 24 * 3600 * 1000` হলে রেকর্ডটি মুছে নতুন অনুরোধ হিসেবে চালান। একটি পরিষ্কারের কাজ (`setInterval`, Server বন্ধে `clearInterval`) পুরোনো রেকর্ড মুছে Memory সীমিত রাখে। Test-এ একটি নকল ঘড়ি: প্রথম অনুরোধ, ঘড়ি ২৪ ঘণ্টা + ১ ms এগিয়ে, একই Key—এবার 201 ও নতুন ID। Database-এ এটি একটি `expires_at` Column ও নিয়মিত `DELETE`।

সমাধান ১১ — URL: `POST /v1/leads/:id/notes` (Note Lead-এর অংশ, তাই Sub-resource); সফল হলে 201 ও `Location: /v1/leads/:id/notes/:noteId`; Lead দেখা না গেলে 404; লেখা না থাকলে বা ১,০০০ অক্ষরের বেশি হলে 400। অনুমতি: Lead যিনি দেখতে পারেন, তিনিই Note যোগ করতে পারেন (`canSee`), কারণ Note ইতিহাস যোগ করে, কিছু বদলায় না। Service: `addNote(user, leadId, text)` — `get(user, leadId)` দিয়ে অনুমতি, তারপর `{ id, author: user.id, text, createdAt }` যোগ। Note কখনো সম্পাদনা নয়, শুধু যোগ (Audit-এর জন্য)।

সমাধান ১২

sqlite-repository.mjs · JavaScript · অংশ ১ / ২ (Part 1 of 2) — চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

export function createSqliteLeadRepository(file = ':memory:') {
  const db = new DatabaseSync(file);
  db.exec(`CREATE TABLE IF NOT EXISTS leads (
    id TEXT PRIMARY KEY, version INTEGER NOT NULL, data TEXT NOT NULL)`);
  const sql = {
    count: db.prepare('SELECT COUNT(*) AS n FROM leads'),
    insert: db.prepare('INSERT INTO leads (id, version, data) VALUES (?, ?, ?)'),
    get: db.prepare('SELECT data FROM leads WHERE id = ?'),
    replace: db.prepare(
      'UPDATE leads SET version = ?, data = ? WHERE id = ? AND version = ?'),
    remove: db.prepare('DELETE FROM leads WHERE id = ?'),
    after: db.prepare('SELECT data FROM leads WHERE id > ? ORDER BY id'),
  };
  return {
    async nextId() {
      return `L-${String(sql.count.get().n + 1).padStart(5, '0')}`;
    },
    async insert(lead) {
      sql.insert.run(lead.id, lead.version, JSON.stringify(lead));
    },
    async get(id) {
      const row = sql.get.get(id);
      return row ? JSON.parse(row.data) : null;
    },
```

```
    async replace(lead, expected) {
      const data = JSON.stringify(lead);
      const { changes } = sql.replace.run(lead.version, data, lead.id, expected);
      return changes === 1;
    },
    async remove(id) {
      return sql.remove.run(id).changes === 1;
    },
    async list({ where = () => true, after = '', limit = 20 }) {
      const rows = sql.after.all(after)
        .map((r) => JSON.parse(r.data)).filter(where);
      return { items: rows.slice(0, limit), hasMore: rows.length > limit };
    },
  };
}

// ...
```

sqlite-repository.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

const repo = createSqliteLeadRepository();
const lead = { id: await repo.nextId(), name: 'Mina', status: 'new', version: 1 };
await repo.insert(lead);
const first = await repo.replace({ ...lead, status: 'contacted', version: 2 }, 1);
const stale = await repo.replace({ ...lead, status: 'lost', version: 2 }, 1);
console.log(first, stale, (await repo.get('L-00001')).status);
```

Output

```
true false contacted
```

`UPDATE ... WHERE id = ? AND version = ?`: Version মিললে একটি সারি বদলায় (`changes === 1`), না মিললে শূন্য—দ্বিতীয় হালনাগাদটি ধরা পড়েছে। এই Adapter-এর দুটি দুর্বলতা জেনে রাখুন: `nextId` সারির সংখ্যা গুনে ID দেয়, তাই মুছে ফেলার পর পুরোনো ID আবার আসতে পারে (অধ্যায় ২১-এর ত্রুটি!), এবং `list` সব সারি পড়ে Memory-তে Filter করে। পরের অধ্যায়ে Database-এর নিজস্ব হাতিয়ার—`AUTOINCREMENT` বা Sequence, `WHERE`-এ Filter, Index—দিয়ে এগুলোর সমাধান।

২৫.১৭ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. নতুন Resource সফলভাবে তৈরি হলে সঠিক Status?

ক) 200

খ) 201

গ) 204

ঘ) 302

প্রশ্ন ২. Express-এ Error-handling Middleware কীভাবে চেনা যায়?

ক) চারটি Parameter: `(err, req, res, next)`

খ) নাম `errorHandler`

গ) সবার আগে `app.use`

ঘ) `async` Function

প্রশ্ন ৩. Express 4-এ `async` Route-এ ধরা-না-পড়া Error-এর ফল?

ক) স্বয়ংক্রিয় 500

খ) স্বয়ংক্রিয় 404

গ) Client উত্তর পান না এবং `unhandledRejection` ঘটে

ঘ) কিছুই না

প্রশ্ন ৪. Token থাকা একজন Agent অন্য Agent-এর Lead মুছতে চাইলে LeadDesk কী দেয়?

ক) 401

খ) 500

গ) 200

ঘ) 404, কারণ সেই Lead তাঁর জন্য দৃশ্যমানই নয়

প্রশ্ন ৫. `Idempotency-Key` -এর উদ্দেশ্য?

ক) একই `POST` Retry-তে ডুপ্লিকেট রেকর্ড তৈরি না হওয়া

খ) Authentication

গ) Cache

ঘ) Rate Limit

প্রশ্ন ৬. `If-Match: "v1"` পাঠানো হালনাগাদে Server-এর Version ২ হলে?

ক) 200

খ) 412

গ) 409

ঘ) 404

প্রশ্ন ৭. Offset-এর বদলে Cursor Pagination-এর সুবিধা?

ক) কম Code

খ) Token লাগে না

গ) পড়ার মাঝে নতুন সারি যোগ হলেও ডুপ্লিকেট বা বাদ পড়ে না, এবং বড় তালিকায় দ্রুত

ঘ) সব পাতা একসঙ্গে

প্রশ্ন ৮. `/ready` Endpoint কখন 503 দেয়?

ক) কখনো না

খ) Token ভুল হলে

গ) Database-এ তথ্য না থাকলে

ঘ) Server বন্ধ হওয়ার প্রক্রিয়ায়, যাতে নতুন Traffic না আসে

প্রশ্ন ৯. Log-এ কী রাখা উচিত নয়?

ক) `Authorization` Header ও ব্যক্তিগত তথ্য

খ) Request ID

গ) Status Code

ঘ) সময়

প্রশ্ন ১০. Route, Service ও Repository আলাদা রাখার মূল লাভ?

ক) দ্রুত HTTP

খ) ব্যবসার নিয়ম HTTP ও Database থেকে স্বাধীন, তাই আলাদা Test ও বদলানো যায়

গ) কম File

ঘ) Express লাগে না

২৫.১৮ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–ক · ৬–খ · ৭–গ · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	সঙ্গে `Location` Header।
২	ক	Express Parameter-এর সংখ্যা দেখে চেনে।
৩	গ	উদাহরণ ০৩-এ দেখানো; সমাধান `wrap` বা Express 5।
৪	ঘ	অস্তিত্ব লুকানো; নিজের দেখা Lead মুছতে চাইলে 403।
৫	ক	Retry-তে একই Key, Server আগের ফল ফেরত দেয়।
৬	খ	Precondition Failed: আবার পড়ে নিন।
৭	গ	Cursor বলে "এই ID-র পর থেকে", Offset নয়।
৮	ঘ	Load Balancer Traffic সরিয়ে নেয়।
৯	ক	Log-ও ফাঁস হয়; তাই শুধু নির্বাচিত Field।
১০	খ	Service Express ছাড়াই Test করা যায়।

২৫.১৯ অধ্যায়ের সারসংক্ষেপ

- Server-এ প্রতিটি অনুরোধ অবিশ্বস্ত; একটি ভুল সবার তথ্যে প্রভাব ফেলে।
- REST: Resource-ভিত্তিক URL, অর্থপূর্ণ Method ও Status, `/v1`-এ সংস্করণ।
- Express: Route, Router, Middleware-এর ক্রম; চার-Parameter-এর Error Handler; Express 4-এ `async` Route-এর জন্য `wrap`।
- Route পাতলা, নিয়ম Service-এ, সংরক্ষণ Repository-তে (`async` চুক্তি)।
- Bearer Token (Hash-এ সংরক্ষিত), Role, মালিকানা; অন্যের তথ্যে 404।
- Cursor Pagination, Idempotency-Key, Version ও `If-Match`—Network-এর অনিশ্চয়তা ও একসঙ্গে কাজের মধ্যে তথ্য সঠিক রাখে।
- Request ID, Structured Log, Rate Limit, Health/Ready, Graceful Shutdown।
- একই Repository-চুক্তির পেছনে Memory বা Database।

পরের অধ্যায়ে Database-এর গভীরে: Schema ও Constraint, SQL, Transaction ও Rollback, Index ও Query Plan, Migration এবং Backup।

২৫.২০ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
REST	Resource ও HTTP Method-ভিত্তিক API-র নকশা।

পরিভাষা	অর্থ
Middleware	অনুরোধ-শৃঙ্খলের একটি ধাপ `(req, res, next)`।
Error Handler	`(err, req, res, next)` Middleware।
Safe Method	Server-এর অবস্থা বদলায় না।
Idempotent	একবার বা বহুবার পাঠালে একই ফল।
Bearer Token	`Authorization` Header-এ পাঠানো পরিচয়ের প্রমাণ।
Cursor Pagination	"এই অবস্থানের পর থেকে" পাতা।
Idempotency-Key	Retry-কে নিরাপদ করার Client-প্রদত্ত Key।
Optimistic Concurrency	Version মিলিয়ে হালনাগাদ, Lock ছাড়া।
ETag	একটি Resource-এর সংস্করণের চিহ্ন।
Request ID	একটি অনুরোধকে সব Log-এ চেনার ID।
Graceful Shutdown	চলমান কাজ শেষ করে বন্ধ হওয়া।

২৫.২১ আরও পড়ুন

- **Express — Guide:** `https://expressjs.com/en/guide/routing.html` — Routing ও Middleware।
- **Express — Error handling:** `https://expressjs.com/en/guide/error-handling.html` — Express 4 ও 5-এর পার্থক্যসহ।
- **MDN — HTTP response status codes:** `https://developer.mozilla.org/en-US/docs/Web/HTTP/Status` — প্রতিটি Status-এর অর্থ।
- **Stripe — Idempotent requests:** `https://docs.stripe.com/api/idempotent_requests` — Idempotency-Key-এর বাস্তব ব্যবহার।
- **Node.js — HTTP server.close():** `https://nodejs.org/api/http.html#serverclosecallback` — Graceful Shutdown-এর ভিত্তি।